



Vanishing and Exploding Gradients Problems in Deep Learning

Last Updated : 15 Jan, 2026

To train deep neural networks effectively, managing the Vanishing and Exploding Gradients Problems is important. These issues occur during backpropagation when gradients become too small or too large, making it difficult for the model to learn properly. Both problems directly affect the model's convergence and overall performance.

Vanishing Gradient Problem

Vanishing gradients occur when gradients become extremely small during backpropagation, causing early layers to learn very slowly or stop learning. During backpropagation the gradient of the loss L with respect to a weight w_i in layer i is calculated using the chain rule:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \cdot \frac{\partial a_{n-1}}{\partial a_{n-2}} \dots \frac{\partial a_1}{\partial w_i}$$

where

- L : Loss function.
- w_i : Weight parameter in the layer.
- a_n : Activation output of layer.
- $\frac{\partial L}{\partial w_i}$: Gradient of loss with respect to weight.

When activation functions like [Sigmoid](#) or [Tanh](#) are used, their derivatives are less than 1. Repeated multiplication through layers causes the gradient to vanish as it moves backwards, making the lower layers unable to learn.

Exploding Gradient Problem

Exploding gradients occur when gradients grow too large during backpropagation, leading to unstable weight updates and divergence in loss. When derivatives or weights are greater than 1, their repeated multiplication across layers leads to exponential growth.

$$\prod_{i=1}^n \frac{\partial a_i}{\partial a_{i-1}} \longrightarrow \infty$$

The gradient update rule in gradient descent is:

$$w_{t+1} = w_t - \eta \cdot \frac{\partial L}{\partial w_t}$$

where

- w_i : Current weight value at time step t .

- η : Learning rate.
- $\frac{\partial L}{\partial w_t}$ Gradient of loss with respect to weight.
- w_{t+1} : Updated weight after applying gradient descent.

If $\frac{\partial L}{\partial w_t}$ is too large weight updates become massive causing the model loss to oscillate or diverge.

Why do the Gradients Vanish or Explode

- **Activation Functions:** Sigmoid or Tanh have small derivatives that shrink gradients.
- **Weight Initialization:** Too small or too large weights cause vanishing or exploding gradients.
- **Deep Networks:** Many layers multiply gradients repeatedly leading to instability.
- **Learning Rate:** High learning rate or unscaled inputs can make gradients explode.

Techniques to Fix Vanishing and Exploding Gradients

Vanishing and exploding gradients make training deep neural networks difficult. The following methods help stabilize gradient flow and improve learning

1. Proper Weight Initialization

Choosing the right weight initialization keeps gradients balanced during backpropagation.

- **Xavier Initialization:** Keeps activation variance consistent across layers to stabilize gradients.
- **Kaiming Initialization:** Scales weights for ReLU to preserve signal strength and prevent gradient decay.

2. Use Non Saturating Activation Functions

Sigmoid and Tanh can shrink gradients. Using ReLU or its variants prevents vanishing gradients:

- **ReLU:** Basic rectified linear unit.
- **Leaky ReLU:** Allows small gradients for negative inputs.
- **ELU / SELU:** Helps maintain self normalizing properties.

3. Apply Batch Normalization

Normalizes layer inputs to have zero mean and unit variance, stabilizing gradients and accelerating convergence.

4. Gradient Clipping

Limits gradients to a maximum threshold to prevent them from exploding and destabilizing training.

Step-By-Step Implementation

Here we compare how gradients behave in deep neural networks using Sigmoid and ReLU activations to visualize the vanishing gradient problem through loss curves.

Step 1 : Import Required Libraries

- **numpy**: For numerical and array operations.
- **matplotlib**: For plotting graphs and visualizations.
- **Sequential**: Builds neural networks layer by layer.
- **train_test_split**: split data into training and testing sets
- **tensorflow**: build and train deep neural networks

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```



Step 2: Create a Simple Dataset

- Generates a binary classification dataset
- Keeps the data simple to isolate gradient behavior
- Prevents data complexity from hiding gradient issues

```
np.random.seed(42)

X = np.random.randn(2000, 2)
y = (X[:, 0] + X[:, 1] > 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```



Step 3: Define a Deep Neural Network

- Uses many layers to amplify vanishing gradient effects
- Accepts an activation function as input
- Keeps architecture identical for fair comparison

```
def build_model(activation, layers=20):
    model = Sequential()
    model.add(tf.keras.Input(shape=(2,)))
    model.add(Dense(32, activation=activation))

    for _ in range(layers - 1):
        model.add(Dense(32, activation=activation))

    model.add(Dense(1, activation='sigmoid'))

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='binary_crossentropy'
    )
    return model
```

Step 4: Train Model with Sigmoid Activation



- Sigmoid has small derivatives
- Gradients shrink as they move backward

- Early layers learn very slowly

```
sigmoid_model = build_model('sigmoid', layers=20)

sigmoid_history = sigmoid_model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    verbose=0
)
```

Step 5: Train Model with ReLU Activation

- ReLU is non-saturating
- Preserves gradient magnitude
- Enables faster and more stable learning

```
relu_model = build_model('relu', layers=20)

relu_history = relu_model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    verbose=0
)
```

Step 6: Plot Training Loss Curves

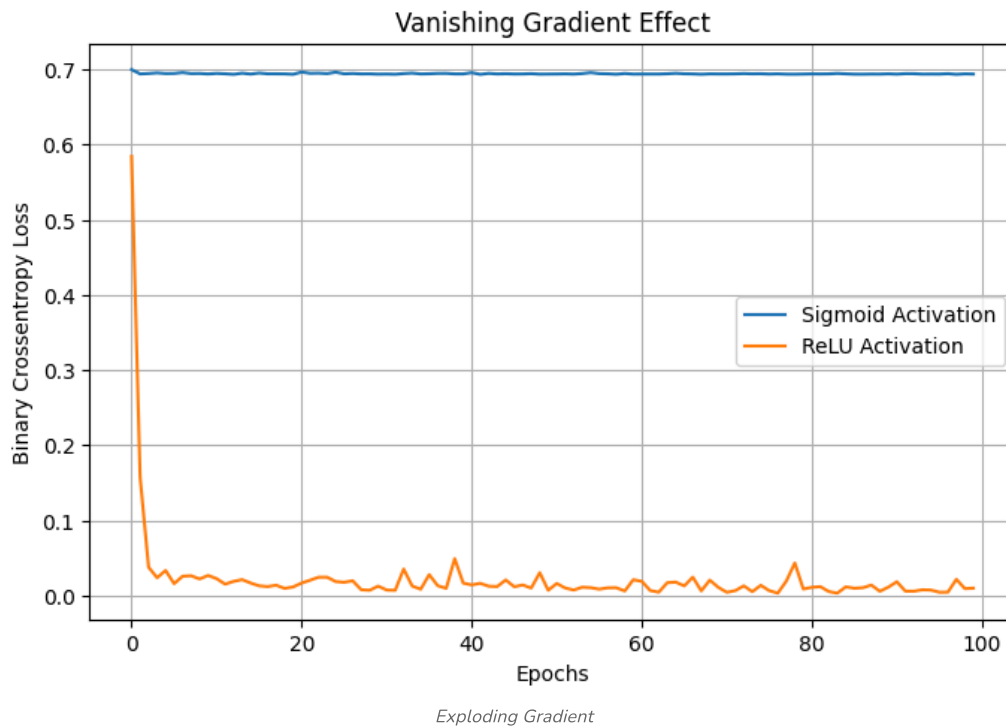
Practice Problems C C++ Java Python JavaScript Data Science Machine Learning Courses Linux De  Sign In

- Rapid loss decrease healthy gradients

```
plt.figure(figsize=(8,5))
plt.plot(sigmoid_history.history['loss'], label='Sigmoid Activation')
plt.plot(relu_history.history['loss'], label='ReLU Activation')
plt.xlabel('Epochs')
plt.ylabel('Binary Crossentropy Loss')
plt.title('Vanishing Gradient Effect')
plt.legend()
plt.grid(True)
plt.show()
```

Loading Playground...

Output:



Interpretation of Results

Sigmoid Activation

- Loss remains almost constant
- Indicates very small gradients
- Demonstrates vanishing gradient problem

ReLU Activation

- Loss decreases rapidly
- Gradients flow effectively
- Avoids vanishing gradient issue

You can download full code file from [here](#).

Impact of Vanishing and Exploding Gradients on RNNs

[Recurrent Neural Networks](#) (RNNs) process sequences step by step, using past outputs as inputs for future steps which makes them particularly sensitive to gradient issues during training.

- **Loss of Long-Term Memory:** The model forgets earlier information because tiny gradients fail to carry learning signals backward.
- **Unstable Training:** Large gradients make the network oscillate or diverge instead of converging smoothly.
- **Poor Sequential Performance:** The RNN struggles to capture long range dependencies needed for complex sequence tasks.
- **Difficult Optimization:** Training becomes unstable and highly dependent on learning rate, initialization and regularization choices.

To solve these problems advanced models like [LSTM](#) and [GRU](#) were introduced. They use gating mechanisms to control the flow of information and keep gradients stable during training. Techniques such as gradient clipping, proper weight initialization and normalization are also used to make training smoother and avoid instability.

Challenges

- **Slow Learning:** Vanishing gradients make early layers learn very slowly or not at all.
- **Unstable Training:** Exploding gradients cause erratic weight updates and diverging loss.
- **Long Term Dependencies:** Vanishing gradients limit memory in sequence models.
- **Poor Convergence:** Gradients issues can lead to suboptimal or failed convergence.
- **Hyperparameter Tuning:** Gradient instability requires careful tuning of learning rates and initialization.

Comment

P

paulsu...

10

Article Tags:

Geeks Premier League

Deep Learning

AI-ML-DS

Geeks Premier League 2023

+1 More

Explore

Deep Learning Basics

Neural Networks Basics

Deep Learning Models

Deep Learning Frameworks

Model Evaluation

Deep Learning Projects



Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company

About Us
Legal
Privacy
Policy
Contact Us
Advertise with us
GFG
Corporate Solution
Training Program

Explore

POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials

Programming Languages
DSA
Web
Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses

ML and Data Science
DSA and Placements
Web
Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web
Development
Data Science
CS Subjects

Preparation Corner

Interview Corner
Aptitude
Puzzles
GfG 160
System Design

